

복잡한 운영을 하나의 흐름으로 바꿉니다

레거시, 반복 업무, 데이터 단절을 통합 플랫폼과 자동화로 해결해 온 풀스택 엔지니어입니다.

경력기술서에 담긴 대표 프로젝트의 문제 해결 과정과 설계 판단을 보충하는 자료입니다.

경력기술서 PDF

GitHub

레거시 시스템 통합

15 → 1

외부 정보 조회

10분 → 5초

반복 작업 절감

월 1,200h

데이터 이관

1,000만+

개발에서 플랫폼 책임까지

기술을 바꾼 기록보다, 맡은 문제의 범위가 어떻게 넓어졌는지를 보여드립니다.

2010–2015

ILJIN Global · Software Engineer

제조 현장의 데이터를 시스템으로 연결

생산 데이터 수집과 MES·ERP 업무 시스템을 개발하며 제조 현장의 흐름을 익혔습니다.

2016–2022

ILJIN Global · Project Lead

국내외 공장 MES 고도화를 리드

노후 클라이언트를 현대화하고 국내외 공장의 추적성과 실시간 ERP 연계를 구축했습니다.

2022–2023

KWE Korea · Lead Software Engineer

물류 업무 자동화와 데이터 전환

반복 입력, 외부 연동, 이기종 데이터 이관을 검증 가능한 자동화 흐름으로 바꿨습니다.

2023–현재

KWE Korea · Lead Software Engineer

레거시를 통합 플랫폼으로 전환

15종 운영 시스템을 분석하고 웹·API·Worker·데이터 경계를 갖춘 단일 플랫폼으로 전환하고 있습니다.

02 · FLAGSHIP CASE

15종 레거시 시스템을 단일 플랫폼으로

2023-현재 · 운영을 멈추지 않고 구조를 바꾼 통합 프로젝트

문제

15종 이상의 프로그램과 3종의 DBMS가 각자 업무와 외부 연동을 품고 있어 변경과 장애의 영향 범위를 파악하기 어려웠습니다.

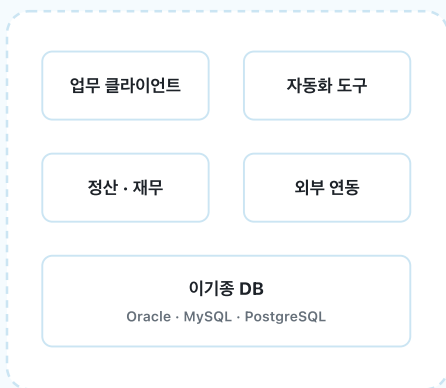
제약

매일 사용하는 운영 시스템이므로 업무를 중단하거나 전체 기능을 한 번에 재작성할 수 없었습니다.

역할

기존 시스템 분석, 목표 구조 설계, 웹·API 구현, 데이터 전환, 단계별 배포와 운영을 주도했습니다.

BEFORE · 15 SYSTEMS



점진적 전환



AFTER · INTEGRATED PLATFORM



실제 시스템명과 연결 정보는 익명화하고, 15종 운영 시스템이 책임과 경계를 가진 플랫폼으로 수렴한 흐름만 표현했습니다.

OUTCOMES

- 15종의 분산된 시스템을 5개 업무 도메인의 단일 웹 플랫폼으로 수렴

- 외부 조회 업무를 건당 10분에서 5초 이내로 단축
- 변경 경계와 배포 계약을 코드와 빌드 단계에서 검증

SUPPORTING EVIDENCE

- **1,000만 건 이상 이기종 데이터 이관과 교차 검증**
Oracle과 PostgreSQL 양쪽 데이터를 자동 비교해 누락과 불일치를 검증했습니다.
- **Express에서 NestJS 모듈 구조로 점진적 전환**
운영 중인 구 API를 유지한 채 도메인 단위로 신규 구조에 이전했습니다.
- **테스트 게이트와 변경 경로 기반 CI/CD 표준화**
실패한 테스트가 배포로 이어지지 않고 변경된 앱만 빌드되도록 구성했습니다.

ENGINEERING DECISIONS

결과보다 판단의 근거를 남겼습니다

문제와 대안을 먼저 적고, 왜 이 선택을 했으며 운영에서 무엇이 달라졌는지 연결합니다.

핵심 판단

한 번에 돌아왔지 않았다

운영 중단 없이 전환했고, 어느 시점에 멈춰도 시스템이 동작하도록 유지했습니다.

문제	레거시 API는 매일 쓰이는 운영 시스템이었습니다. 전면 재작성은 멈출 수 없는 업무를 볼모로 잡는 일이었습니다.
검토	기능 동결 후 일괄 전환도 검토했습니다. 전환 기간 동안 개선 요청을 모두 거절해야 했습니다.
선택	구 API를 살려둔 채 모듈 단위로 신규 API에 이전했습니다. 공용 라이브러리도 구·신 두 벌을 병행하고 신규 기능은 신규 구조에만 넣었습니다.
결과	운영 중단 없이 전환했고, 어느 시점에 멈춰도 시스템이 동작하도록 유지했습니다.

핵심 판단

경계를 규칙이 아니라 타입으로 지켰다

등록을 잊은 채 배포되는 사고를 구조적으로 차단했습니다.

문제	폴더로 계층을 나뉘도 시간이 지나면 도메인이 인프라를 직접 호출합니다. 코드 리뷰만으로 막는 데는 한계가 있었습니다.
검토	린트 규칙으로 import 경로를 막는 방법도 있었습니다. 우회가 쉽고 위반 시점이 늦었습니다.
선택	도메인은 Symbol 포트만 알고 구현체는 모듈 등록에서 주입했습니다. 확장되는 지점은 유형 목록을 키로 가진 레코드 타입으로 두어 새 유형의 핸들러가 빠지면 컴파일되지 않게 했습니다.
결과	등록을 잊은 채 배포되는 사고를 구조적으로 차단했습니다.

핵심 판단

배포 단위가 다른 둘 사이의 계약

계약 위반이 런타임이 아니라 빌드 단계에서 드러납니다.

문제	잡을 넣는 API와 꺼내 쓰는 Worker가 따로 배포됩니다. 페이로드 필드 하나를 바꾸면 배포 순서에 따라 런타임 오류가 발생했습니다.
검토	메시지에 버전을 붙여 양쪽에서 분기하는 방법도 검토했습니다. 분기가 쌓이면 제거하기 어려워집니다.
선택	큐 이름, 잡 타입, 페이로드, 내부 라우트를 한 패키지에 모으고 양쪽이 그것만 참조하게 했습니다. 계약을 바꾸면 두 앱이 함께 컴파일 대상이 됩니다.
결과	계약 위반이 런타임이 아니라 빌드 단계에서 드러납니다.

왜 통합인가

새 외부 시스템을 붙일 때 고쳐야 할 곳이 15곳에서 1곳이 됐습니다.

문제	15종 클라이언트가 각자 DB와 외부 시스템에 직접 붙어 있었습니다. 같은 정산입력 자동화가 업무별로 여러 번 존재했고, 장애가 나면 어디서 시작됐는지 추적하기 어려웠습니다.
검토	레거시를 그대로 두고 연동 게이트웨이만 앞에 세우는 안도 검토했습니다. 단기 비용은 낮지만 중복 로직은 그대로 남습니다.
선택	화면과 API를 하나의 플랫폼으로 수렴시키고, 외부 연동은 어댑터 한 층으로 모았습니다. 데이터 접근 경로를 하나로 만드는 것이 목적이었습니다.
결과	새 외부 시스템을 붙일 때 고쳐야 할 곳이 15곳에서 1곳이 됐습니다.

클린 아키텍처를 프론트에 넣으려다 접었다

화면 수정이 폴더 하나 안에서 끝나고 삭제도 폴더 단위로 수행할 수 있게 됐습니다.

문제	백엔드에서 레이어 분리로 효과를 봤기에 같은 구조를 프론트에 이식했습니다. 그런데 화면 하나를 고치려면 여러 폴더를 오가야 했고, 레이어 대부분이 통과만 하는 껍데기가 됐습니다.
검토	레이어를 유지하고 규칙을 더 촘촘히 만드는 방향도 시도했습니다. 규칙이 늘수록 신규 화면 작성 비용만 올라갔습니다.
선택	업무 화면 90여 개는 화면 자체가 업무 단위였습니다. 화면 폴더 안에 로컬 슬라이스를 두고, 화면을 넘어 재사용되는 것만 도메인 슬라이스로 올렸습니다.
결과	화면 수정이 폴더 하나 안에서 끝나고 삭제도 폴더 단위로 수행할 수 있게 됐습니다.

replica를 늘리자 크론이 두 번 돌았다

Worker 수가 늘어도 잡은 한 번만 실행되고, 한 대가 멈춰도 남은 대가 이어받습니다.

문제	가용성을 위해 Worker를 2대로 늘렸더니 예약 작업이 양쪽에서 동시에 실행되어 집계가 중복 반영됐습니다.
검토	Worker 한 대를 예약 실행 전담으로 고정할 수 있었지만, 그 한 대가 멈추면 전체 예약 실행이 중단됩니다.
선택	실행 직전 Redis에 TTL을 가진 키를 NX로 선점하게 했습니다. 선점에 성공한 한 대만 실행하고 나머지는 넘어갑니다.
결과	Worker 수가 늘어도 잡은 한 번만 실행되고, 한 대가 멈춰도 남은 대가 이어받습니다.

서류를 밖으로 내보낼 수 없었다

서류가 사내망을 벗어나지 않고 스키마를 벗어난 응답은 파싱 단계에서 차단됩니다.

문제	수출입 서류에는 거래처 단가와 신고 정보가 들어 있어 외부 API로 전송할 수 없었습니다.
검토	사람이 계속 입력하거나 규칙 기반 파서를 만들 수 있었지만, 거래처마다 다른 서식을 규칙만으로 감당하기 어려웠습니다.
선택	사내망에 OpenAI 호환 추론 서버를 두었습니다. 출력은 JSON Schema로 제한하고 형식이 정해진 값은 정규식으로 교차 검증하며, 텍스트 품질이 낮은 PDF만 OCR로 넘겼습니다.
결과	서류가 사내망을 벗어나지 않고 스키마를 벗어난 응답은 파싱 단계에서 차단됩니다.

믿지 않고 측정했다

추출 신뢰도를 수치로 비교할 수 있고, 장시간 운영에서도 API 메모리가 안정적으로 유지됩니다.

문제	모델 출력은 매번 같지 않아 잘 되는 것 같다는 느낌만으로 운영에 올릴 수 없었습니다.
검토	샘플 몇 건을 눈으로 확인하는 방식은 실패율이 낮을수록 문제를 발견하기 어렵습니다.
선택	같은 문서를 반복 추출해 결과를 DB에 적재하는 평가 하네스를 만들고 프롬프트나 모델을 바꿀 때마다 실행했습니다. OCR은 자식 프로세스로 분리해 장시간 메모리 증가도 막았습니다.
결과	추출 신뢰도를 수치로 비교할 수 있고, 장시간 운영에서도 API 메모리가 안정적으로 유지됩니다.

외부 8종을 한 경계 뒤로

외부 장애와 사양 변경이 도메인으로 새지 않고, 연동 하나를 교체해도 도메인 코드는 유지됩니다.

문제	공공 통관 API, 상용 EDI 플랫폼, 거래처 전문, 계근대 하드웨어까지 연동 방식이 제각각이었고 각 시스템이 필요할 때마다 직접 연결했습니다.
검토	공동 클라이언트 하나로 추상화하는 방법도 검토했지만 HTTP, FTP, 소켓을 하나로 묶으면 억지 추상화가 됩니다.
선택	추상화를 강요하지 않고 위치만 통일했습니다. 외부와 통신하는 코드는 어댑터 층에 두고 도메인은 포트 인터페이스만 봅니다.
결과	외부 장애와 사양 변경이 도메인으로 새지 않고, 연동 하나를 교체해도 도메인 코드는 유지됩니다.

모노레포에서 바뀐 것만 배포한다

무관한 앱은 실행되지 않고 운영 중인 이미지와 커밋의 관계를 태그로 추적할 수 있습니다.

문제	여러 앱이 한 저장소에 있어 커밋 하나에도 전체가 빌드됐고, 대기 시간이 길며 무관한 앱까지 재배포됐습니다.
검토	앱마다 저장소를 나누면 공용 패키지의 버전 관리 비용이 커집니다.
선택	부모 파이프라인이 변경 경로를 감지해 해당 앱의 자식 파이프라인만 실행했습니다. 빌드가 생성한 커밋 SHA 이미지 태그를 배포가 그대로 사용합니다.
결과	무관한 앱은 실행되지 않고 운영 중인 이미지와 커밋의 관계를 태그로 추적할 수 있습니다.

외부 업무 시스템 입력 자동화

자동 입력 자체보다, 사람이 개입하지 않아도 안전하게 운영되는 흐름에 집중했습니다.

문제

여러 담당자가 문서를 확인하고 같은 데이터를 외부 업무 시스템에 반복 입력하면서 휴먼 에러와 장시간 수작업이 발생했습니다.

역할

문서 검증 규칙, 자동 입력 흐름, 실패 감지와 재처리 도구를 설계하고 구현했습니다.

01

검증

필수값, 합계, 형식과 업무 조건을 입력 전에 확인했습니다.

02

처리

검증을 통과한 데이터만 자동 전송하고 입력 결과를 기록했습니다.

03

예외

실패 원인과 처리 위치를 남기고 담당자가 안전하게 재처리할 수 있게 했습니다.

RESULT

반복 작업을 무인 흐름으로 전환해 월 기준 약 1,200시간의 수작업을 절감했습니다.

04 · EXPERIENCE

경력과 기술을 사용 맥락으로 설명합니다

직함이나 기술 목록보다 어떤 운영 문제에서 무엇을 책임졌는지를 남깁니다.

2022.01-현재

전략개발부 차장 · Lead Software Engineer

KWE Korea

물류 운영 시스템의 풀스택 개발과 운영을 리드하며 통합, 자동화, 데이터 전환과 배포 표준화를 수행했습니다.

- 15종 레거시 시스템을 5개 업무 도메인의 웹 플랫폼으로 전환
- 1,000만 건 이상 이기종 데이터 이관 및 정합성 교차 검증
- NestJS 구조 전환과 GitLab CI/CD·컨테이너 배포 표준화

2010.12-2022.01

MES 개발팀 과장 · Software Engineer

ILJIN Global

제조 현장의 MES와 ERP 연동을 개발하고 국내외 공장 고도화 프로젝트를 리드했습니다.

- VB6 기반 MES를 C#·.NET 서비스 구조로 현대화
- 실시간 ERP 양방향 연동과 제품 단위 LOT 추적 체계 구축
- 일 100만 건 이상 생산 데이터를 분석하는 품질 알림 시스템 개발

Technology in context

Product & Frontend

Next.js · React · TypeScript · Zustand · WinForms

90여 개 업무 화면, 화면 중심 슬라이스, 운영 도구 개발

Backend & Automation

Node.js · NestJS · Express · C# · Redis · BullMQ

도메인 API, 비동기 작업, RPA와 배치 처리

Data & Integration

PostgreSQL · Oracle · MySQL · SAP · gRPC · WCF

이기종 데이터 이관, 정합성 검증, 외부 시스템 연동

Delivery & Operations

GitLab CI/CD · Docker · Docker Swarm · Portainer

테스트 게이트, 변경 경로 기반 빌드, 배포 추적과 롤백

직접 만들고 운영한 제품

지원 직무와 연결되는 개인 프로젝트만 간결하게 정리했습니다.

댓글 필터 · Android & Web

브라우저와 단말에서 댓글 분류·본문 요약을 수행해 서버 비용과 데이터 외부 전송을 줄인 1인 개발 프로젝트입니다.

React Native · Next.js · TypeScript · Transformers.js · WebLLM · Web Worker

Android

Web

개발기

액션독

게임 코어 엔진을 계층별로 설계하고 인증, 글로벌 랭킹, 광고까지 연결해 출시한 3라인 매치 게임입니다.

React Native · Expo · TypeScript · Supabase · AdMob

Google Play

HealthDog

운동 기록·조화·통계와 데이터 동기화, 구독 결제를 구현해 운영 중인 개인 프로젝트입니다.

Google OAuth · Firebase · Supabase · RevenueCat

Google Play